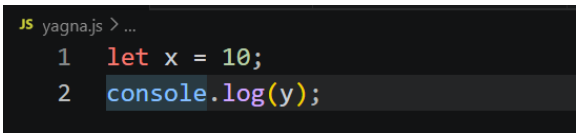
Module 10 – Application Debugging

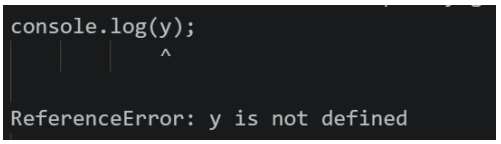
Application debugging is the process of **finding, analyzing, and fixing errors (bugs)** in an application. In React applications, debugging helps developers identify problems in components, state management, event handling, rendering, and performance.

React applications can be debugged using tools such as: Chrome DevTools, React Developer Tools and Visual Studio Code Debugger. These tools allow developers to inspect HTML elements, monitor JavaScript execution, examine variables, and identify runtime errors.

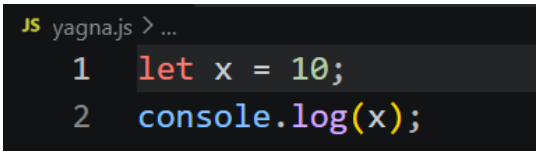
Debugging:

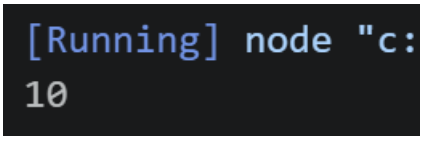
Debugging is the process of Finding errors, understanding why they occur, Fixing them and Testing to ensure they are resolved.





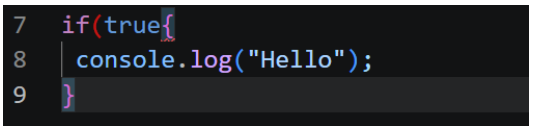
Here variable **y** does not exist.

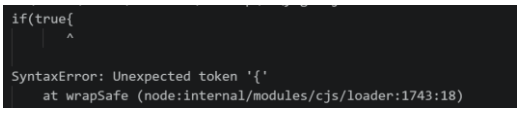




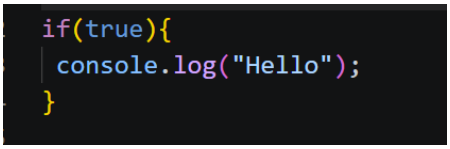
Types of Errors

1. Syntax Errors: These occur when code violates language rules.

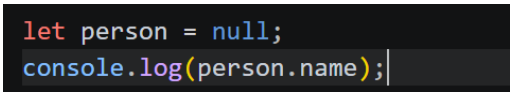


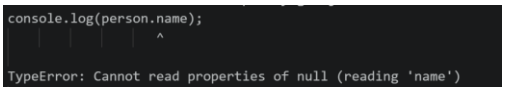


Correct Code:

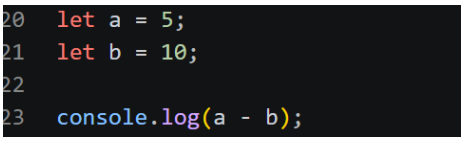


2. Runtime Errors: These occur while the program is executing.





3. Logical Errors: Program executes successfully but gives incorrect output.



Actually expected output is -15 but the output is -5 because wrong operator use

Debugging is Important because it finds bugs quickly, saves development time, improves software quality, prevents application crashes, enhances user experience and improves performance.

Chrome DevTools:

Chrome DevTools is a built-in debugging tool available in Google Chrome.

Open it by Pressing **F12**, **Ctrl + Shift + I** and then Right-click → **Inspect**

It contains several panels like Elements, Console, Sources, Network, Performance, Memory and Application.

1. Elements Panel

The main purpose is to inspect HTML and CSS. Its main uses are View DOM tree, Edit HTML, Modify CSS, Check styles and Find layout issues.

```
yagna.html > html
 1  <!DOCTYPE html>
 2  <html>
 3  <head>
 4      <title>React Demo</title>
 5  </head>
 6  <body>
 7      <div id="root">
 8          <h1>Hello React</h1>
 9      </div>
10  </body>
11  </html>
```

2. Console Panel

The main purpose is to Execute JavaScript and view errors.

```
25  let age = 20;
26  console.log(age);
```

```
20
```

Error example:

```
28  console.log(name);
```

```
console.log(name);
            ^

ReferenceError: name is not defined
```

Some of the console methods are console.log(), console.error(), console.warn(), console.table() and console.clear().

3. Sources Panel:

Its main purpose is to Debug JavaScript. It also includes features like Breakpoints, Step Into, Step Over, Step Out, Call Stack and Watch Variables.

Breakpoint:

A breakpoint pauses program execution at a specific line so you can inspect variables and understand program flow. Placing breakpoint as **return a+b; .**

```
30  function add(a,b){
31      return a+b;
32  }
33
34  let result = add(5,6);
35  console.log(result);
```

```
11
```

Types of Breakpoints:

1. **Line Breakpoint:** Stops at a particular line.

2. **Conditional Breakpoint:** Stops only when a condition is true.

Example

count == 10

Execution pauses only when count becomes 10.

3. **Event Breakpoint:** Pauses when events occur.

Examples like Click, Mouse move and Keyboard input.

4. **Exception Breakpoint:** Stops execution when an error (exception) occurs.

Stepping Through Code:

When paused at a breakpoint, Chrome DevTools and VS Code provide stepping controls:

Step Into (F11): Enters the called function and executes it line by line.

```
function square(x){
    return x*x;
}

square(5);
```

Using **Step Into** opens the square() function and lets you inspect its execution.

Step Over (F10): Executes the current line without entering function calls.

Step Out (Shift + F11): Runs the rest of the current function and pauses after returning to the caller.

Watch Variables: The **Watch** window monitors selected variables during debugging.

Example:

```
44  let price = 100;
45  let tax = 18;
46  let total = price + tax;
```

Watch:

price
tax
total

During execution:

price = 100
tax = 18
total = 118

Local Variables: The **Locals** window automatically shows all variables available in the current scope.

```
function calculate(){

 let a = 10;
 let b = 20;
 let c = a+b;

}
```

Here locals are: a = 10, b = 20, c = 30

DOM (-Document Object Model): The DOM represents an HTML document as a tree of objects that JavaScript can manipulate.

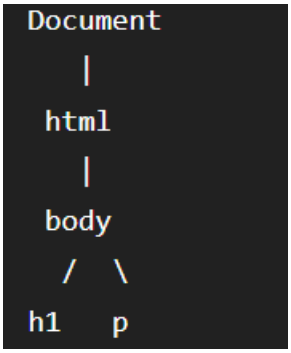
```
<html>
<body>

<h1>Hello</h1>

<p>React</p>

</body>
</html>
```

DOM Tree:



Inspect Element: Inspect Element allows you to View HTML, Edit CSS temporarily, Find missing elements, Debug layouts and Identify incorrect classes or styles.

React Developer Tools: React Developer Tools is a Chrome extension for debugging React applications. It contains Features like View component hierarchy, Inspect props, Inspect state, Edit state, Highlight updates and Measure performance with the Profiler.

```
function App(){
   return <h1>Hello React</h1>;
}
```

Component Tree:

App

└── h1

Source Maps: Source maps allow browsers to map compiled or bundled JavaScript back to the original TypeScript or JSX source code, making debugging much easier. Instead of seeing transpiled JavaScript, you debug the original .tsx or .ts files.

Visual Studio Code Debugger: VS Code provides an integrated debugger for React applications. Some common features are Breakpoints, Watch Variables, Call Stack, Debug Console and Variable Inspection.

Basic workflow:

1. Open the project.
2. Go to the **Run and Debug** view.
3. Create or use a launch.json configuration.
4. Start debugging (F5).
5. Set breakpoints in your React or TypeScript files.
6. Inspect variables, call stack, and watch expressions as execution pauses.

Advantages of Debugging:

- Detects bugs early.
- Improves application stability.
- Reduces development time.
- Enhances code quality.

- Simplifies maintenance.
- Improves user experience.